Day 2 :DBMS Notes: Normalization, Transactions and ACID Properties

1. Normalization in DBMS

Normalization is the process of organizing data in a database so that **data redundancy is reduced** and problems related to insertion, deletion, and updating of data are avoided.

The main purpose of normalization is to divide a large table into smaller related tables and connect them using keys.

Why do we need Normalization?

Suppose we store student and course information repeatedly in one large table. If the same teacher, course, or department information appears for many students, the same data gets repeated again and again.

This causes:

- Data redundancy
- Wastage of storage
- Difficulty in updating data
- Inconsistent data
- Insertion anomaly
- Deletion anomaly
- Update anomaly

Anomalies in DBMS

Insertion Anomaly:
It occurs when we cannot insert some information without inserting some other unnecessary information.

For example, suppose course details and student details are stored together. If a new course starts but no student has enrolled yet, we may not be able to store the course because student information is required.

Deletion Anomaly:
It occurs when deleting one piece of information accidentally deletes some other important information.

For example, if only one student is enrolled in a course and we delete that student's record, the information about that course may also be lost.

Update Anomaly:
It occurs when the same information exists in multiple rows and we have to update it everywhere.

For example, if a teacher's phone number appears in 50 rows, we must update all 50 rows. If some rows are not updated, inconsistent data is created.

Normalization solves these problems by dividing the data into separate related tables.

---

2. First Normal Form – 1NF

A table is in **First Normal Form** when:

- Every column contains only a single value.
- There are no multiple or repeating values inside one cell.
- Each record can be uniquely identified.

For example, storing:

Student = Rahul
Phone = 9876, 8765

is not in 1NF because one column contains multiple phone numbers.

It should be stored as separate records or in a separate phone table.

The main purpose of 1NF is to make every value **atomic**, meaning indivisible.

---

3. Second Normal Form – 2NF

A table is in **Second Normal Form** when:

1. It is already in 1NF.
2. There is no **partial dependency**.

Partial dependency occurs when a non-key attribute depends only on a part of a composite primary key instead of the complete key.

For example, suppose the primary key is:

StudentID + CourseID

and the table contains:

StudentName, CourseName, Marks

Here:

StudentName depends only on StudentID.

CourseName depends only on CourseID.

Marks depends on both StudentID and CourseID.

Therefore, StudentName and CourseName create partial dependency.

To convert it into 2NF, we can create separate tables for students, courses, and enrollments.

The main purpose of 2NF is to remove **partial dependency**.

---

4. Third Normal Form – 3NF

A table is in **Third Normal Form** when:

1. It is already in 2NF.
2. There is no **transitive dependency**.

Transitive dependency occurs when a non-key attribute depends on another non-key attribute.

Suppose we have:

StudentID, StudentName, DepartmentID, DepartmentName

Here:

StudentID determines DepartmentID.

DepartmentID determines DepartmentName.

Therefore:

StudentID indirectly determines DepartmentName.

This is called transitive dependency.

To solve this, Department information should be stored in a separate table.

The main purpose of 3NF is to ensure that non-key attributes depend **only on the primary key**.

---

5. BCNF – Boyce-Codd Normal Form

BCNF is a stronger version of 3NF.

A table is in BCNF when:

For every functional dependency X → Y, X must be a super key.

In simple words, the attribute determining another attribute should be capable of uniquely identifying a record.

BCNF removes some unusual dependencies that may still remain after 3NF.

Generally:

1NF removes repeating values.

2NF removes partial dependency.

3NF removes transitive dependency.

BCNF ensures every determinant is a candidate key or super key.

Transactions in DBMS

A **transaction** is a group of one or more database operations that are treated as a single unit of work.

A transaction may contain operations such as:

- INSERT
- UPDATE
- DELETE
- SELECT

A common example is a bank transfer.

Suppose ₹1000 is transferred from Account A to Account B.

The database performs two important operations:

1. Subtract ₹1000 from Account A.
2. Add ₹1000 to Account B.

Both operations must happen together.

If money is deducted from Account A but not added to Account B because the system crashes, the database becomes incorrect.

Therefore, DBMS treats these operations as a single transaction.

Transaction Commands

COMMIT

COMMIT permanently saves the changes made by a transaction.

Example:

```
UPDATE account
SET balance = balance - 1000
WHERE id = 1;

COMMIT;
```

After COMMIT, the changes become permanent.

ROLLBACK

ROLLBACK cancels the changes made by a transaction that have not yet been committed.

Example:

```
UPDATE account
SET balance = balance - 1000
WHERE id = 1;

ROLLBACK;
```

The account balance returns to its previous value.

SAVEPOINT

SAVEPOINT creates a temporary point inside a transaction.

We can roll back to that particular point instead of cancelling the entire transaction.

Example:

SAVEPOINT s1;

Then:

ROLLBACK TO s1;

States of a Transaction

A transaction can move through different states.

Active State

The transaction is currently executing its operations.

Partially Committed State

All operations have been executed, but the changes are not yet permanently stored.

Committed State

The transaction has completed successfully and all changes are permanently stored.

Failed State

An error occurs and the transaction cannot continue.

Aborted State

The transaction is rolled back and the database returns to its previous consistent state.

ACID Properties

ACID properties ensure that database transactions are executed safely and reliably.

ACID stands for:

A – Atomicity
C – Consistency
I – Isolation
D – Durability

---

1. Atomicity

Atomicity means **either the complete transaction happens or nothing happens**.

It follows the idea:

All or Nothing.

Consider transferring ₹1000 from Account A to Account B.

Two operations occur:

Deduct ₹1000 from A.

Add ₹1000 to B.

If the second operation fails, the first operation should also be cancelled.

The DBMS performs a rollback.

Therefore, partial transactions are not allowed.

---

2. Consistency

Consistency means a transaction must take the database from **one valid state to another valid state**.

Database rules and constraints must remain satisfied.

For example, suppose the total amount between Account A and Account B is ₹10,000.

Before transfer:

Total = ₹10,000

After transferring ₹1000 from A to B:

Total must still remain ₹10,000.

The transaction should not create invalid or incorrect database data.

Therefore, consistency maintains the correctness of the database.

---

3. Isolation

Isolation means multiple transactions executing at the same time should not interfere with each other.

Each transaction should behave as if it is executing alone.

For example, suppose two people simultaneously try to withdraw money from the same bank account.

If both transactions read the same old balance and update it independently, incorrect results may occur.

DBMS uses concurrency control and locking mechanisms to prevent such problems.

Isolation protects transactions from the intermediate changes made by other transactions.

---

4. Durability

Durability means once a transaction has been successfully committed, its changes will remain permanent.

Even if:

- The system crashes
- Electricity goes off
- The server restarts

the committed data should not be lost.

For example, if ₹1000 has successfully been transferred and the transaction is committed, restarting the bank server should not reverse the transaction.

DBMS uses techniques such as logs, backups, and permanent storage to provide durability.

Simple ACID Example

Consider an ATM withdrawal of ₹500.

Suppose the initial account balance is ₹5000.

After withdrawal, the balance should become ₹4500.

Atomicity:
Either the complete ₹500 withdrawal happens or it does not happen.

Consistency:
The account follows all database rules and the correct balance becomes ₹4500.

Isolation:
Another simultaneous transaction should not interfere with this withdrawal.

Durability:
After the withdrawal is committed, the ₹4500 balance remains saved even if the server crashes.

Quick Revision

Normalization is used to reduce redundancy and remove anomalies.

1NF removes repeating or multiple values.

2NF removes partial dependency.

3NF removes transitive dependency.

BCNF ensures that every determinant is a super key.

A transaction is a collection of database operations treated as one logical unit.

COMMIT saves changes permanently.

ROLLBACK cancels uncommitted changes.

SAVEPOINT creates a point within a transaction to which we can return.

ACID properties make transactions reliable.

Atomicity means **all or nothing**.

Consistency means **database rules remain correct**.

Isolation means **transactions do not interfere with each other**.

Durability means **committed data remains permanently saved**.